

MINERÍA DE DATOS

Maximiliano Ojeda

muojeda@uc.cl



IIC-2433



Recordatorio

Componentes Serie de Tiempo

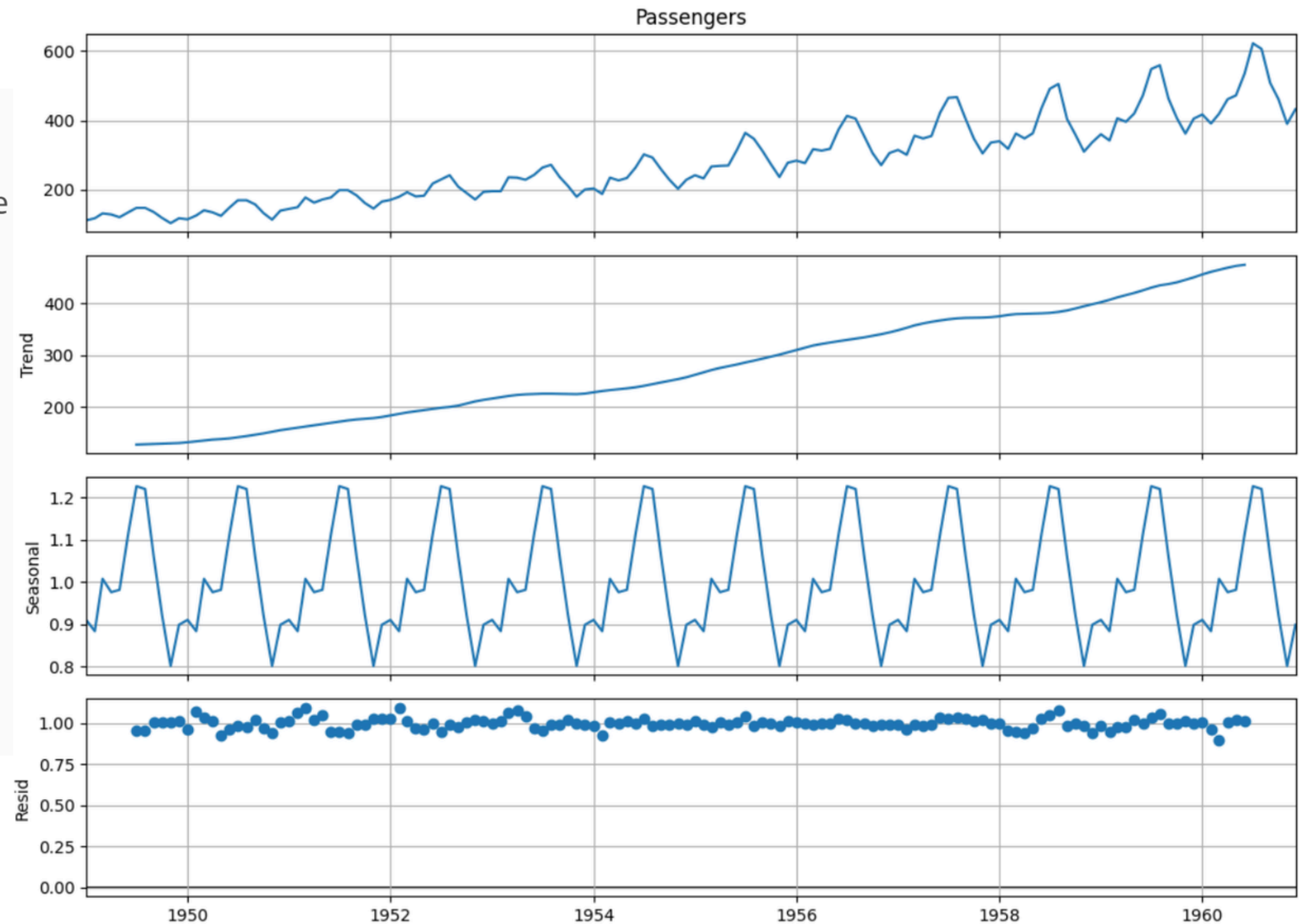
```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
```

```
url = "airline-passengers.csv"
df = pd.read_csv(url)
df["Month"] = pd.to_datetime(df["Month"])
df = df.set_index("Month")
df = df.asfreq("MS")
df.head()
```

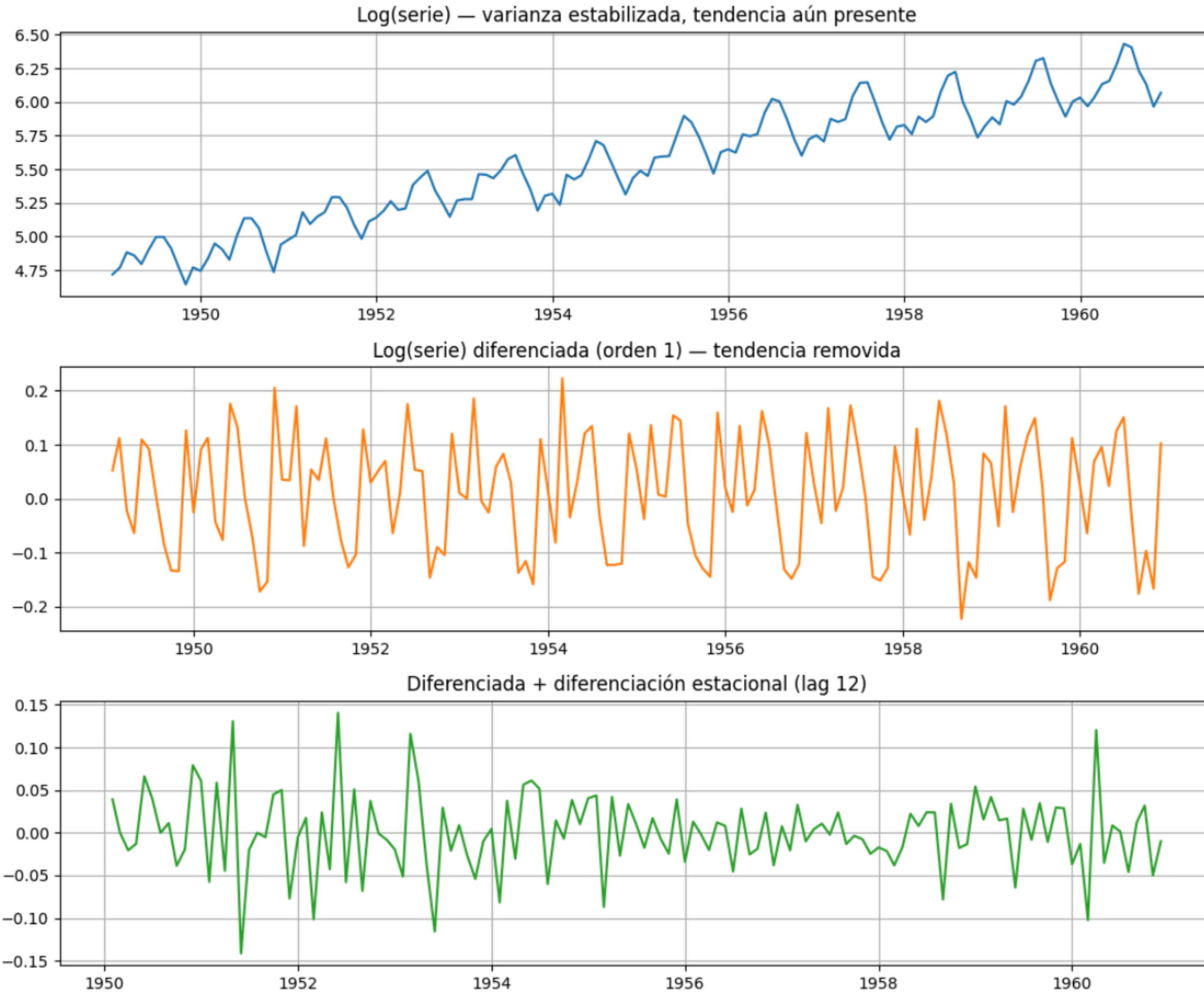
```
resultado = seasonal_decompose(df["Passengers"],
                               model="multiplicative", period=12)
```

```
fig = resultado.plot()
fig.set_size_inches(11, 8)
plt.show()
```

Passengers	
Month	
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121



Transformaciones para estacionariedad



Autocorrelación

La autocorrelación mide **qué tan relacionado está un valor de la serie con sus propios valores pasados**

ACF (Autocorrelation Function)

Mide la correlación entre y_t y y_{t-k} para distintos rezagos k (lag 1, lag 2, lag 3, ...), sin controlar por los rezagos intermedios:

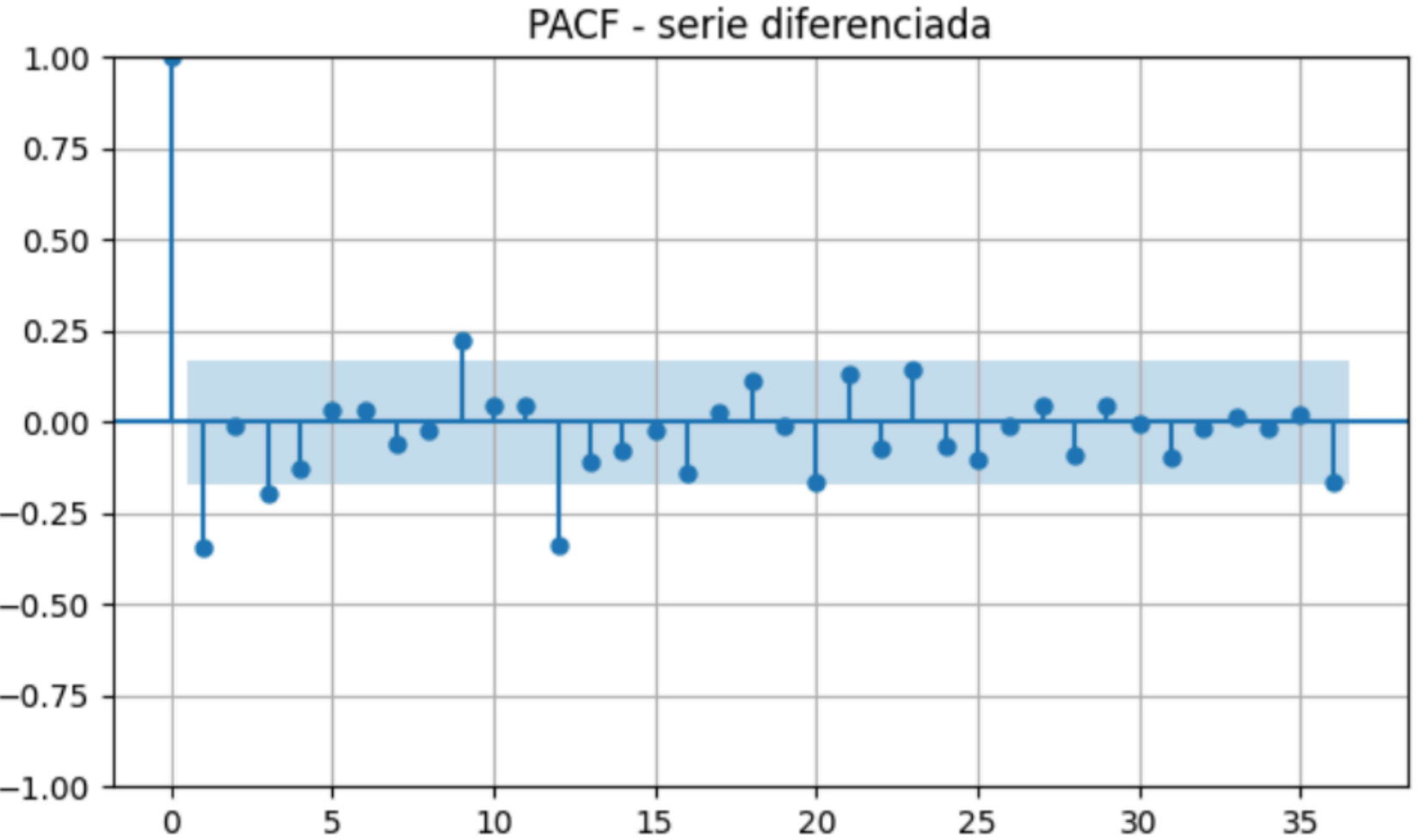
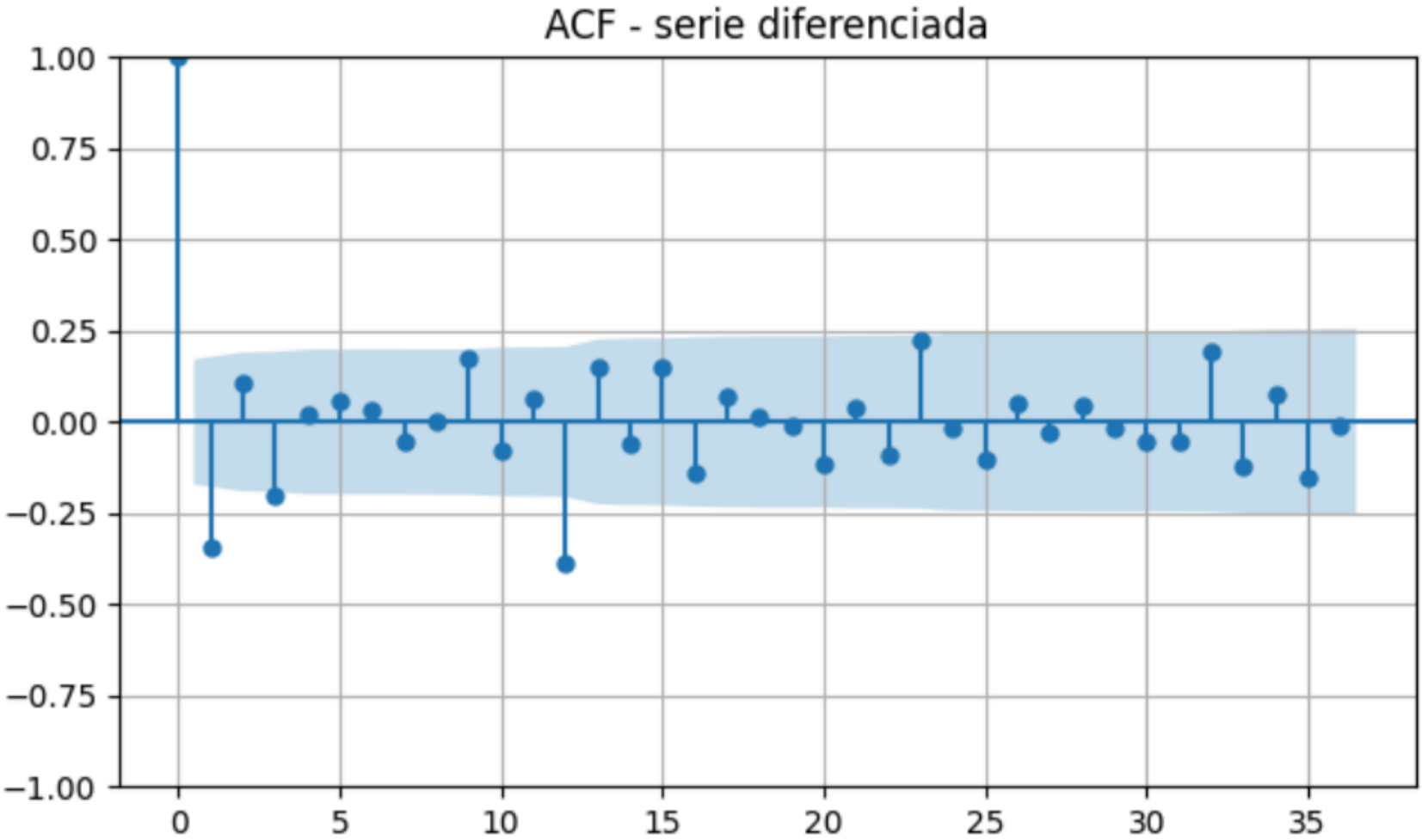
$$\rho_k = \frac{Cov(y_t, y_{t-k})}{Var(y_t)}$$

Si hay correlación en el lag 2, parte de esa correlación puede venir "prestada" indirectamente a través del lag 1. La ACF no distingue eso, mezcla el efecto directo e indirecto

PACF (Partial Autocorrelation Function)

Mide la correlación entre y_t y y_{t-k} , pero **removiendo el efecto de los rezagos intermedios**. Es la correlación "neta" o "directa" en el lag k . **La PACF pregunta:** "¿ y_{t-2} aporta información adicional sobre y_t una vez que ya conozco y_{t-1} ?"

Autocorrelación





Modelo ARIMA

ARIMA

ARIMA (Autoregressive Integrated Moving Average) es un algoritmo estadístico clásico utilizado para analizar y predecir datos de series temporales **basándose exclusivamente en sus propios valores y errores históricos.**

AR

Autoregressive for looking
at past observations

I

Differencing (or Integrated) for
handling changes in data patterns

MA

Moving Average for
considering error terms

Modelo AR (Autoregresivo)

El valor actual de la serie se explica como una combinación lineal de sus propios valores pasados, más un término de error aleatorio

Fórmula general, AR(p):

$$y_t = \phi_1 y_{t-1} + \phi_2 y_{t-2} + \dots + \phi_p y_{t-p} + \varepsilon_t$$

p = el orden del modelo → cuántos rezagos hacia atrás “mira”

ε_t = ruido blanco (aleatorio, media cero, no correlacionado en el tiempo)

Caso más simple AR(1):

$$y_t = \phi_1 y_{t-1} + \varepsilon_t$$

Ej “La temperatura de hoy se parece a la de ayer y anteayer...” hay “memoria” directa de los valores anteriores.

Modelo MA (Media Móvil)

El valor actual de la serie se explica como una combinación lineal de errores pasados, no de valores pasados de la serie

Fórmula general, MA(q):

$$y_t = \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \dots + \theta_q \varepsilon_{t-q}$$

q = el orden del modelo → cuántos shocks pasados todavía influyen en el valor actual

$\varepsilon_t, \varepsilon_{t-1}, \dots$ = la misma secuencia de ruido blanco, pero acá es el objeto que se modela directamente

Diferencia clave con AR

- En AR, el efecto de un shock se propaga indefinidamente (se multiplica por ϕ una y otra vez en cada periodo siguiente. Los shocks se acumulan o se diluyen gradualmente)
- En MA, el efecto de un shock desaparece exactamente después de q periodos. No hay “cola” de decaimiento, es un corte abrupto por construcción matemática

Ej. Un shock de oferta (una noticia inesperada) afecta el precio de un producto por un par de días y después su efecto directo desaparece por completo, no se acumula para siempre.

Combinación ARMA

Un modelo ARMA(p,q) junta los dos mecanismos anteriores: el valor actual depende tanto de sus propios valores pasados como de shocks pasados

Fórmula general, ARMA(p,q):

$$y_t = \underbrace{\phi_1 y_{t-1} + \dots + \phi_p y_{t-p}}_{\text{parte AR}} + \varepsilon_t + \underbrace{\theta_1 \varepsilon_{t-1} + \dots + \theta_q \varepsilon_{t-q}}_{\text{parte MA}}$$

Por qué combinarlos

- En la vida real, muchas series muestran ambos patrones a la vez: dependencia directa de valores pasados (AR) y efectos de shocks que se disipan rápido (MA)
- Con solo AR o solo MA a veces se necesitan muchísimos parámetros para ajustar bien la serie; combinando ambos, frecuentemente se logra el mismo ajuste con menos parámetros totales. Un modelo más simple y más fácil de interpretar

ARIMA: agregando la “I”

Problema:

Casi ninguna serie real es estacionaria de entrada. Por ejemplo, AirPassengers tiene **tendencia** clarísima (y estacionalidad)

Solución:

ARIMA(p, d, q) = aplicar **ARMA(p, q)** sobre la serie **diferenciada d veces**, en vez de sobre la serie original

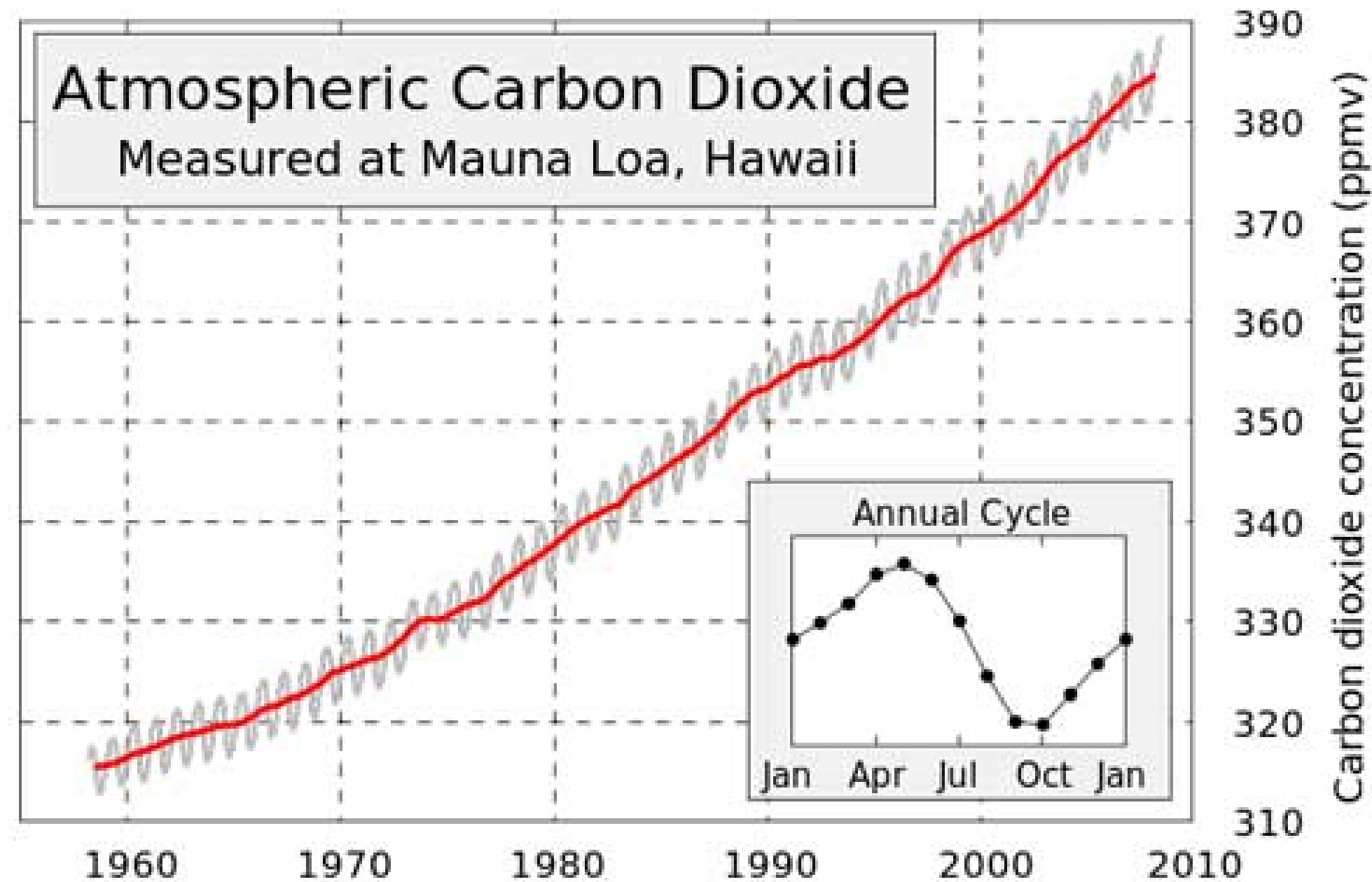
La “I” significa *Integrated*. El nombre viene de que, para volver de la serie diferenciada a la serie original, hay que "integrar" (sumar/acumular) las diferencias, la operación inversa a diferenciar

Model	Equivalente	Descripción
ARIMA(1,0,0)	AR(1)	Simple autoregressive model
ARIMA(0,0,1)	MA(1)	Simple moving average model
ARIMA(0,1,0)	Random walk	Price follows a random walk
ARIMA(1,1,1)	-	One of the most commonly used specifications

ARIMA con d=1 (o 2) ya estabiliza la tendencia. En AirPassegners, **incluso después de diferenciar, quedaba sin explicar en el lag 12**, la estacionalidad todavía no está resuelta. Ahí entra SARIMA.

Ejemplo CO₂ atmosférico

Desde 1958, Charles David Keeling midió la concentración de CO₂ en Mauna Loa, Hawái, casi ininterrumpidamente. Muestra una tendencia ascendente clarísima con una estacionalidad anual perfecta (el CO₂ baja en verano boreal porque las plantas absorben más, sube en invierno)



ARIMA statsmodels

```
import statsmodels.api as sm
from statsmodels.tsa.arima.model import ARIMA

co2 = sm.datasets.co2.load_pandas().data
serie = serie.interpolate(method="linear")

train = serie[serie.index < "1998-01-01"]
test = serie[serie.index >= "1998-01-01"]

modelo_arima = ARIMA(train, order=(2, 1, 1)).fit()

forecast_arima = modelo_arima.get_forecast(steps=len(test))
pred_arima = forecast_arima.predicted_mean
intervalo_arima = forecast_arima.conf_int()

# plot()...
```

```

=====
Dep. Variable:                co2      No. Observations:                478
Model:                    ARIMA(2, 1, 1)      Log Likelihood                -472.169
Date:                Sat, 26 Sep 2026      AIC                        952.338
Time:                23:24:56      BIC                        969.008
Sample:                03-01-1958      HQIC                       958.892
                        - 12-01-1997
Covariance Type:                opg
=====

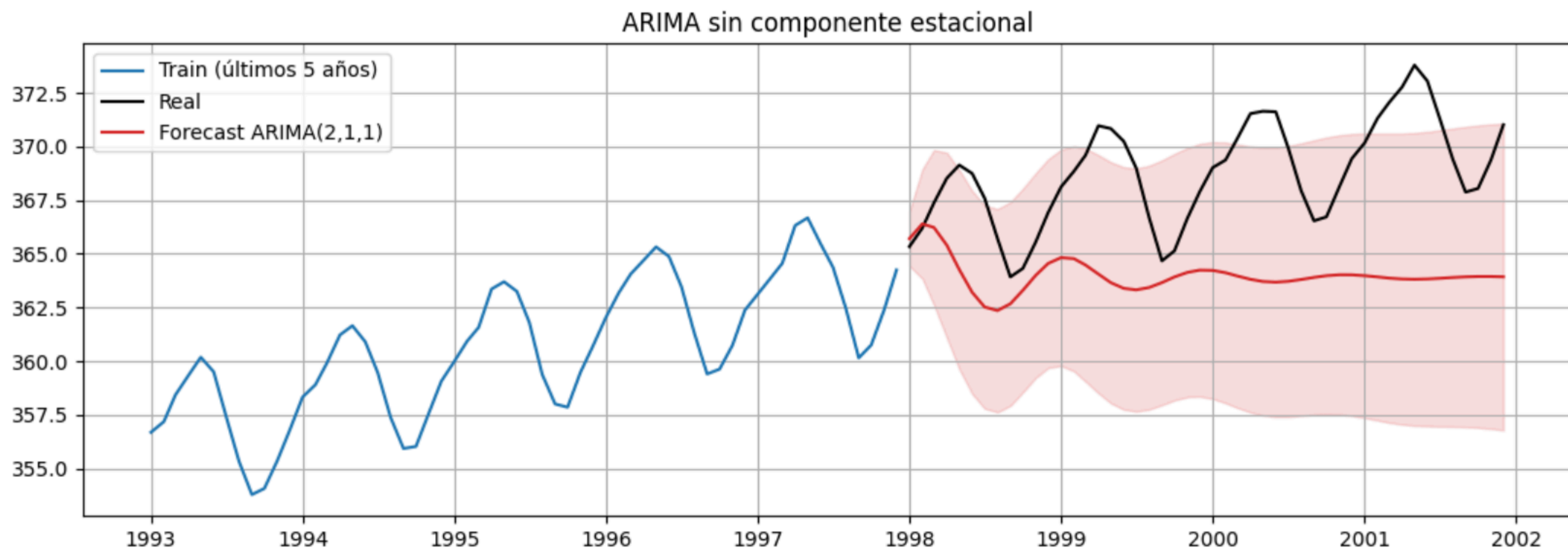
```

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	1.5413	0.029	52.463	0.000	1.484	1.599
ar.L2	-0.8366	0.033	-25.342	0.000	-0.901	-0.772
ma.L1	-0.8129	0.039	-21.061	0.000	-0.889	-0.737
sigma2	0.4219	0.026	16.519	0.000	0.372	0.472

```

=====
Ljung-Box (L1) (Q):                4.47      Jarque-Bera (JB):                16.88
Prob(Q):                0.03      Prob(JB):                0.00
Heteroskedasticity (H):                1.17      Skew:                0.45
Prob(H) (two-sided):                0.31      Kurtosis:                3.16
=====

```



SARIMA

ARIMA modela tendencia y dependencia de corto plazo, pero no captura estacionalidad explícita. No tiene ningún mecanismo dedicado a “esto se repite cada 12 meses”

- $SARIMA(p, d, q)(P, D, Q)_s$: agrega un segundo set de órdenes. Igual en estructura a (p,d,q), pero aplicado sobre el ciclo estacional de periodo s
- s = la longitud del periodo estacional: s=12 para datos mensuales con estacionalidad anual, s=7 para datos diarios con estacionalidad semanal

Parámetro	Qué captura	De dónde sale
p, d, q	Dependencia de corto plazo	PACF / ADF / ACF
P, D, Q	Dependencia estacional	Mismo diagnóstico, pero mirando lags múltiplos de s
s	Periodo del ciclo	Conocido de antemano por el tipo de datos (12, 7, 4, etc)

SARIMA

```
import statsmodels.api as sm
from statsmodels.tsa.arima.model import ARIMA

co2 = sm.datasets.co2.load_pandas().data
serie = serie.interpolate(method="linear")

train = serie[serie.index < "1998-01-01"]
test = serie[serie.index >= "1998-01-01"]

modelo_sarima = SARIMAX(train,
                        order=(2, 1, 1),
                        seasonal_order=(1, 1, 1, 12)).fit()

forecast_sarima = modelo_sarima.get_forecast(steps=len(test))
pred_sarima = forecast_sarima.predicted_mean
intervalo_sarima = forecast_sarima.conf_int()

# plot()...
```

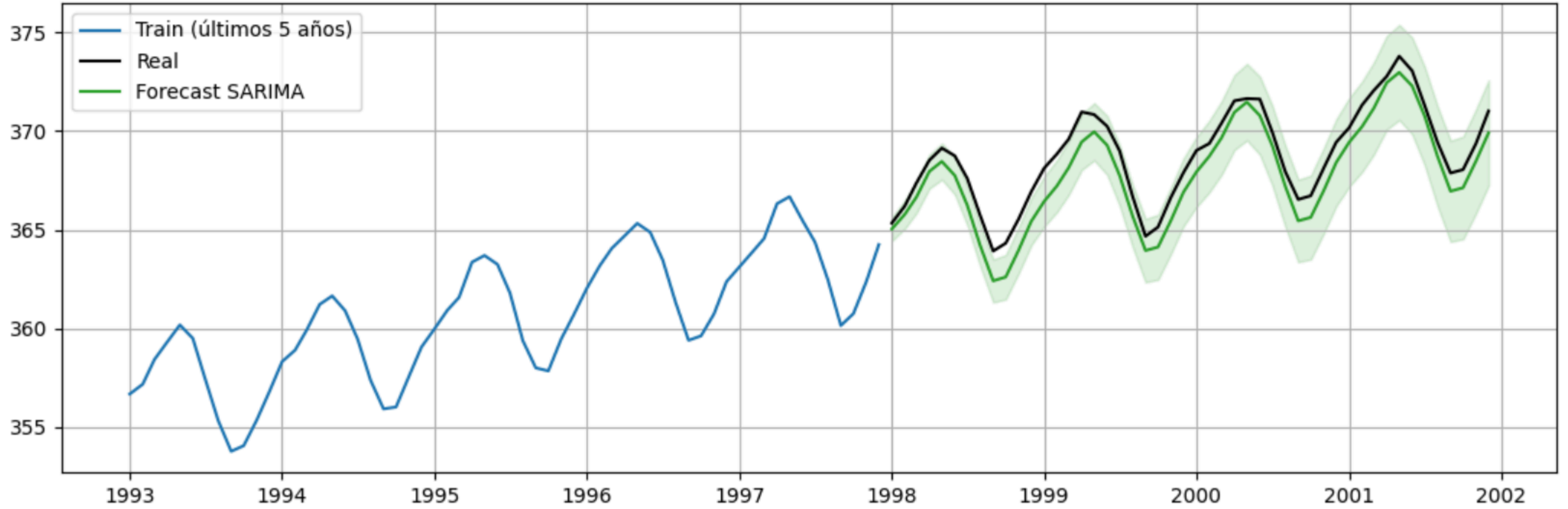
```

=====
SARIMAX Results
=====
Dep. Variable:          co2      No. Observations:          478
Model:                 SARIMAX(2, 1, 1)x(1, 1, 1, 12)  Log Likelihood          -117.603
Date:                  Sat, 26 Sep 2026      AIC                   247.206
Time:                  23:24:56             BIC                   272.058
Sample:                03-01-1958           HQIC                  256.988
                    - 12-01-1997

Covariance Type:      opg
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
ar.L1          0.3770      0.119      3.176      0.001      0.144      0.610
ar.L2          0.0595      0.072      0.821      0.412     -0.083      0.202
ma.L1         -0.7268      0.112     -6.476      0.000     -0.947     -0.507
ar.S.L12       0.0448      0.055      0.808      0.419     -0.064      0.153
ma.S.L12      -0.8595      0.035    -24.602      0.000     -0.928     -0.791
sigma2         0.0939      0.006     15.503      0.000      0.082      0.106
=====
Ljung-Box (L1) (Q):          0.00      Jarque-Bera (JB):          0.19
Prob(Q):                    1.00      Prob(JB):              0.91
Heteroskedasticity (H):      0.85      Skew:                  0.02
Prob(H) (two-sided):        0.30      Kurtosis:              3.09
=====

```

SARIMA(2,1,1)(1,1,1,12)





Prophet

¿Qué es Prophet?

Desarrollado por Meta (2017), pensado para **forecasting a gran escala en industria** (miles de series simultáneas)

- Enfoque distinto a ARIMA: en vez de modelar dependencia estadística entre observaciones (AR/MA), descompone la serie directamente en componentes interpretables y los ajusta por separado

Modelo Aditivo

$$y(t) = g(t) + s(t) + h(t) + \varepsilon_t$$

donde $g(t)$ = tendencia, $s(t)$ = estacionalidad, $h(t)$ = efecto de feriados/eventos especiales



Por qué es popular en la práctica

- Robusto a outliers y datos faltantes. No requiere preprocesarlos a mano antes de ajustar (a diferencia de SARIMA)
- Detecta automáticamente puntos de quiebre en la tendencia. No asume que la tendencia es una sola línea recta o exponencial fija todo el tiempo
- Requiere mucho menos ajuste manual de parámetros que elegir

Prophet

```
from prophet import Prophet

df_prophet = pd.DataFrame({"ds": serie.index, "y": serie.values})
train_prophet = df_prophet[df_prophet["ds"] < "1998-01-01"]
test_prophet = df_prophet[df_prophet["ds"] >= "1998-01-01"]

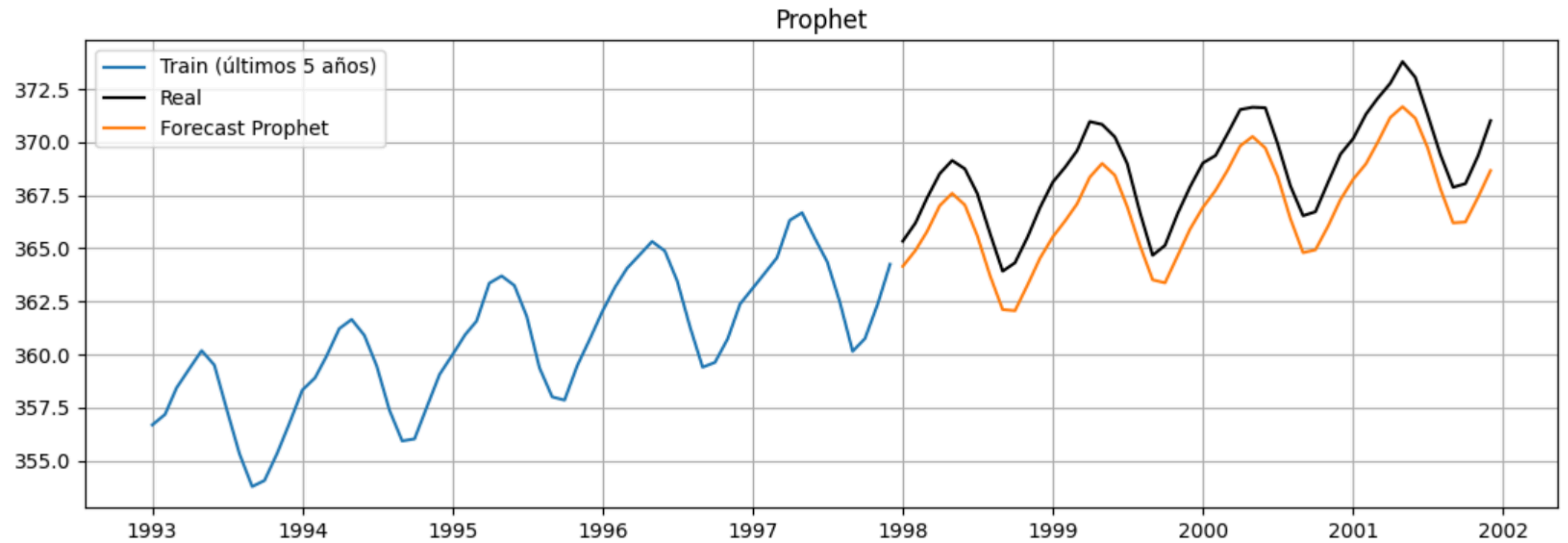
modelo_prophet = Prophet(yearly_seasonality=True,
                          weekly_seasonality=False, daily_seasonality=False)
modelo_prophet.fit(train_prophet)

futuro = modelo_prophet.make_future_dataframe(periods=len(test_prophet), freq="MS")
forecast_prophet = modelo_prophet.predict(futuro)

# Nos quedamos solo con las fechas del periodo de test
pred_prophet = forecast_prophet.set_index("ds").loc[test.index, "yhat"]

#....
plt.show()
```

Prophet





Métricas

Métricas

Elegir modelo sin usar el test

$$AIC = 2k - 2 \ln(\hat{L})$$

$$BIC = k \ln(n) - 2 \ln(\hat{L})$$

Métricas de error de pronóstico (MAE, RMSE, MAPE)

Métrica	Fórmula	Cómo se lee
MAE	$\frac{1}{n} \sum e_t $	Error promedio, en las unidades de la serie
RMSE	$\sqrt{\frac{1}{n} \sum e_t^2}$	También en unidades, pero castiga más los errores grandes
MAPE	$\frac{100}{n} \sum \left \frac{e_t}{y_t} \right $	Error en %, comparable entre series de distinta escala

MINERÍA DE DATOS

Maximiliano Ojeda

muojeda@uc.cl



IIC-2433